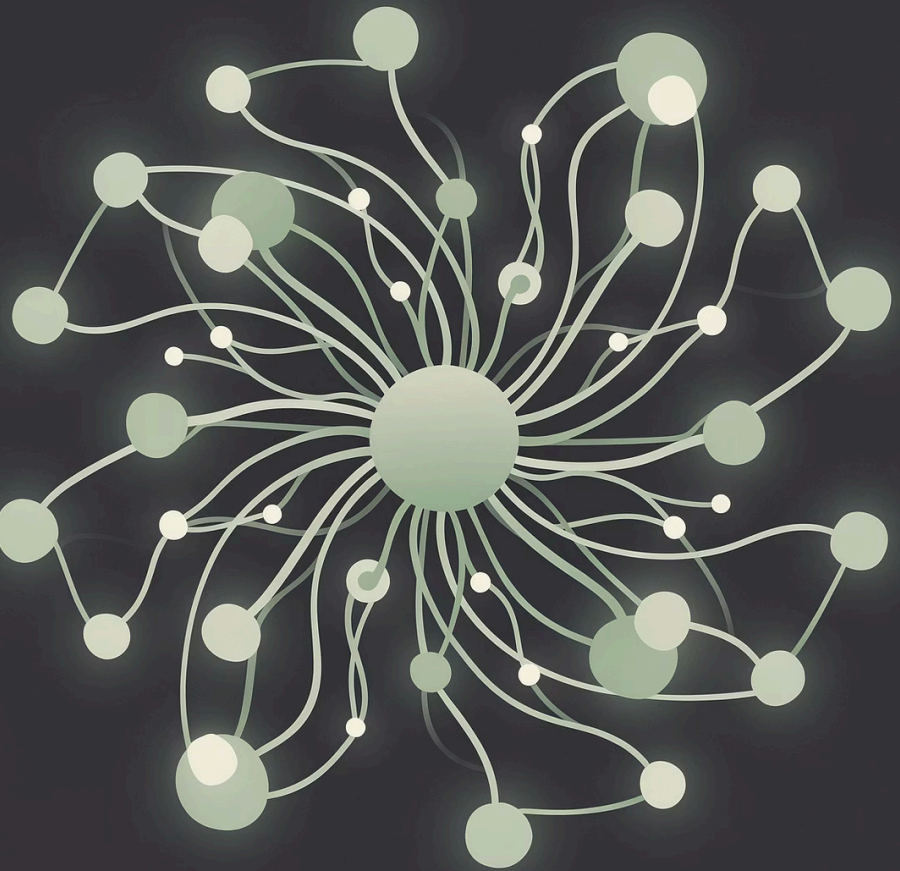




UNLOX

CHAPTER 21

Reinforcement Learning Fundamentals

From the RL paradigm to Q-learning — a comprehensive guide to how agents learn to make decisions through trial, error, and reward.

UNLOCK

OVERVIEW

What We'll Cover

This module walks through the foundational concepts of reinforcement learning, building from high-level intuition to the mathematical framework and a hands-on algorithm.

01

The RL Paradigm

Understand the agent-environment loop, reward signals, and how RL differs from supervised learning.

02

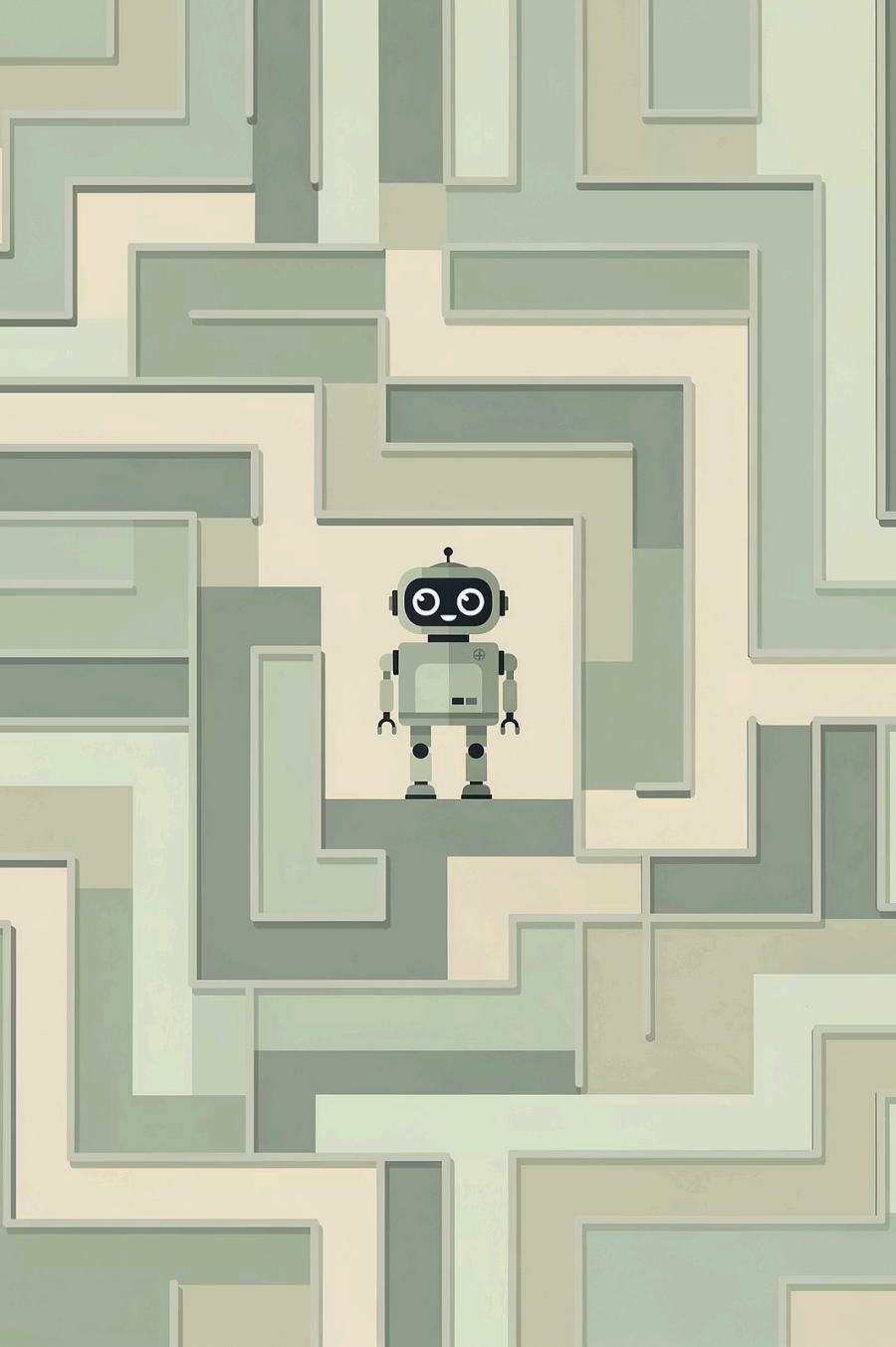
Markov Decision Processes

Learn the formal mathematical framework — states, actions, transitions, and the Markov property.

03

Q-Learning & OpenAI Gym

Implement a classic off-policy algorithm and practice with standardized RL environments.



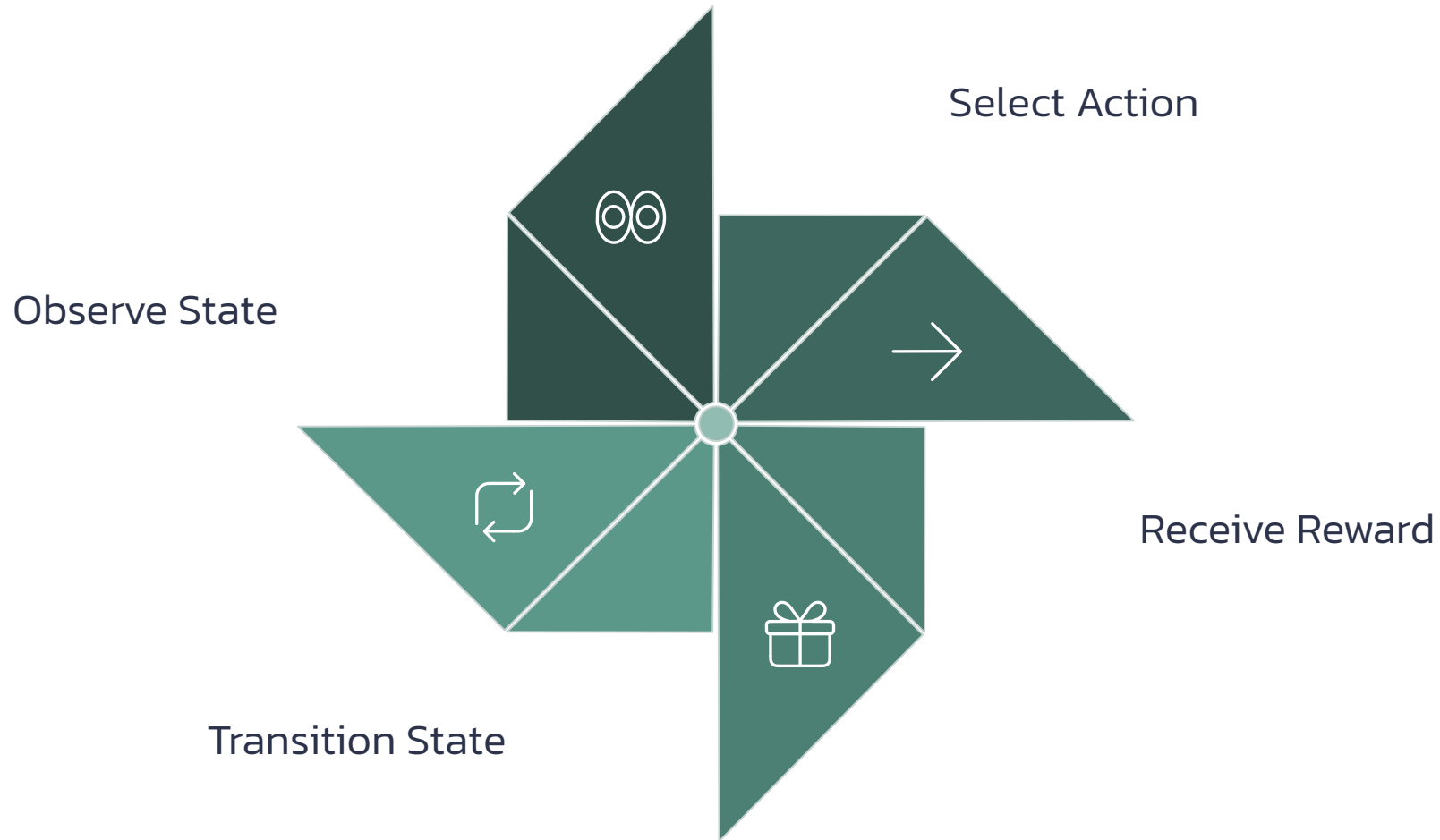
UNLOCK SECTION 21.1

The Reinforcement Learning Paradigm

Reinforcement learning is a **goal-oriented framework** where an **agent** learns to take actions in an **environment** to maximize a cumulative **reward**. Unlike supervised learning — which relies on pre-labeled datasets — the agent discovers effective strategies through **trial and error**, receiving scalar feedback signals after each action. Over time, the agent refines its behavior to select actions that yield the greatest long-term return.

The Agent-Environment Interaction

At every time step, the agent observes the current **state**, selects an **action**, receives a **reward**, and transitions to a new state. This loop repeats continuously as the agent gathers experience.



The agent's objective is to learn an optimal **policy** — a mapping from every possible state to the best action — that maximizes the **expected cumulative reward** over an episode or the agent's lifetime.

UNLOCK

COMPARISON

RL vs. Supervised vs. Unsupervised Learning

Understanding where reinforcement learning sits relative to other machine learning paradigms clarifies its unique strengths and challenges.

Supervised Learning

Learns from labeled input-output pairs. The model minimizes prediction error against known correct answers. No exploration is needed.

Unsupervised Learning

Discovers hidden structure in unlabeled data — clusters, distributions, or latent features. There is no explicit feedback signal at all.

Reinforcement Learning

Learns through interaction and delayed reward signals. Must balance **exploration** of unknown actions with **exploitation** of known good strategies.

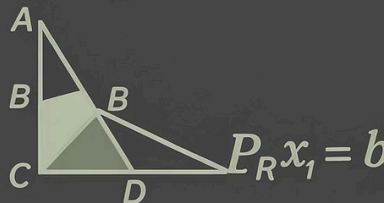
Markov Decision Processes (MDPs)

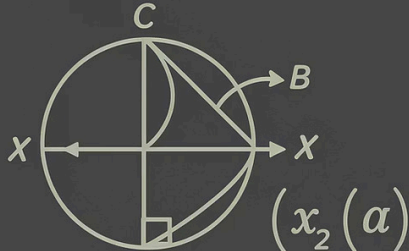
The mathematical backbone of most RL problems is the **Markov Decision Process**. An MDP provides a rigorous framework for modeling sequential decision-making in environments where outcomes are partly random and partly under the agent's control. The critical insight — known as the **Markov property** — is that the probability of reaching the next state depends *only* on the current state and action, not on the entire history of prior states.

$$\frac{d^{22}}{x} = d^2 + \frac{b}{\kappa_1 = \alpha_2}$$

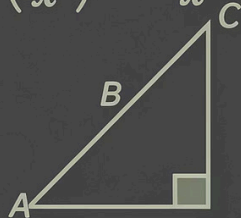
$$\alpha^2 x = (d^2 - b)$$

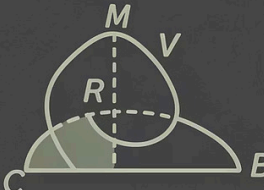
$$Ag^2 = \alpha_1 + (y + k) = d)^3$$

$$x_{-1} = \frac{A + A - f^2 x}{b} \rightarrow \frac{\alpha^2}{a}$$


$$P_R x_1 = b$$


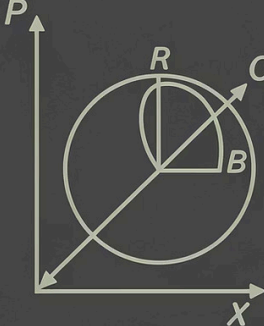
$$(x_2(a))^3$$

$$\left(\frac{2}{x}\right) = \frac{\alpha=0}{x} + (x)^3$$


$$R \int x \frac{x}{2} \left[\frac{x}{4} \right]_1^2 +_2 (\alpha x) + \frac{Pb}{x}$$


$$\int x_1 + \int \alpha \frac{r}{c} x)^2 b \frac{\alpha_3 - b}{x} +$$

$$R = \frac{l}{l_8} + x^{2 \times 2}$$

$$\frac{t^2}{x^3} = \frac{b_1 - k}{c} x$$


$$E^3 x_1 + \alpha^2 + (b - x_1) b = \alpha^2 \frac{\mathcal{L}}{l}$$

The Four Pillars of an MDP

Every MDP is formally defined by four essential components that together capture the full dynamics of the decision problem.



States (S)

The set of all possible situations the agent can find itself in. A state encodes everything the agent needs to make a decision — for example, the position of pieces on a game board.



Actions (A)

The set of all possible moves available to the agent at any given state. Actions can be discrete (left, right) or continuous (steering angle, throttle).



Transition Function (P)

The probability $P(s'|s, a)$ of transitioning to state s' given the agent is in state s and takes action a . This captures environment dynamics and stochasticity.



Reward Function (R)

The scalar signal $R(s, a, s')$ the agent receives after transitioning. The agent's goal is to maximize the expected sum of these rewards over time.

UNLOCK

KEY CONCEPT

The Markov Property Explained

The Markov property is the assumption that makes MDPs tractable.

Formally, it states:

| *"The future is conditionally independent of the past, given the present."*

In mathematical terms, this means:

$$P(s_{t+1}|s_t, a_t) = P(s_{t+1}|s_1, a_1, s_2, a_2, \dots, s_t, a_t)$$

This simplification is powerful — it means the agent only needs to consider the **current state** to make optimal decisions, not the entire trajectory of past states. This dramatically reduces the computational complexity of finding good policies.

Why It Matters

Without the Markov property, the agent would need to remember every past state and action — an exponentially growing history.

With it, the agent can focus entirely on the present, making efficient algorithms like Q-learning and dynamic programming possible.

Many real-world problems approximate this property well enough for RL to succeed.

Q-Learning: Learning Action Values

Q-learning is one of the most foundational and widely used reinforcement learning algorithms. The "Q" stands for **quality** — it quantifies how good a particular action is when taken in a particular state. The algorithm learns a function $Q(s, a)$ that estimates the **expected cumulative future reward** for taking action a in state s and then following the optimal policy thereafter. Crucially, Q-learning is an **off-policy** method, meaning it can learn the optimal policy even while following an exploratory one.



UNLOCK

ALGORITHM

The Q-Learning Update Rule

At the heart of Q-learning is a simple but powerful iterative update. After taking action a in state s , observing reward r , and arriving in next state s' , the Q-value is updated:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$



Learning Rate (α)

Controls how much new information overrides old values. A high α means faster learning but more instability; a low α means slow, steady convergence.



Discount Factor (γ)

Determines how much the agent values future rewards relative to immediate ones. A γ near 1 makes the agent far-sighted; near 0 makes it greedy.



Max Q-Value

The term $\max Q(s', a')$ represents the best possible future value from the next state — the agent bootstraps its estimates using its own predictions.

UNLOCK

PROCESS

How Q-Learning Works Step by Step

The Q-learning algorithm follows a structured loop that repeats across many episodes until the Q-table converges to optimal values.



Initialize Q-Table

Create a table with all state-action pairs set to zero (or small random values).



Choose Action

Use an ϵ -greedy strategy: explore randomly with probability ϵ , exploit best known action otherwise.



Observe & Update

Execute the action, receive reward and new state, then update $Q(s, a)$ using the update rule.



Repeat & Converge

Iterate across episodes. Gradually reduce ϵ so the agent shifts from exploring to exploiting.

UNLOCK

TOOLKIT

OpenAI Gym: Your RL Playground

OpenAI Gym (now maintained as **Gymnasium**) is the standard toolkit for developing, testing, and benchmarking reinforcement learning algorithms. It provides a unified API to interact with a wide variety of environments — from simple text-based grids to physics-simulated control tasks.

The API is elegantly simple: call `env.reset()` to start an episode, `env.step(action)` to advance one time step, and receive back the new observation, reward, and a done flag. This consistent interface lets you swap environments without changing your algorithm code.

CartPole

Balance a pole on a moving cart — a classic control benchmark.

FrozenLake

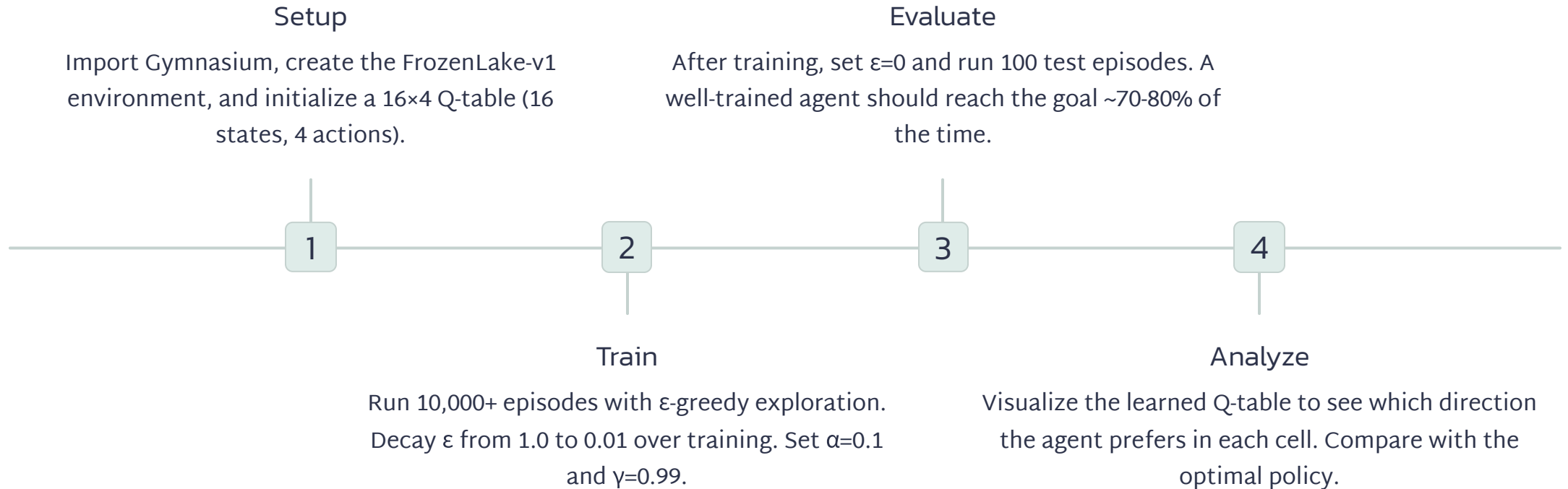
Navigate a grid world while avoiding holes — ideal for tabular Q-learning.

MountainCar

Build momentum to push a car up a steep hill — tests exploration strategies.

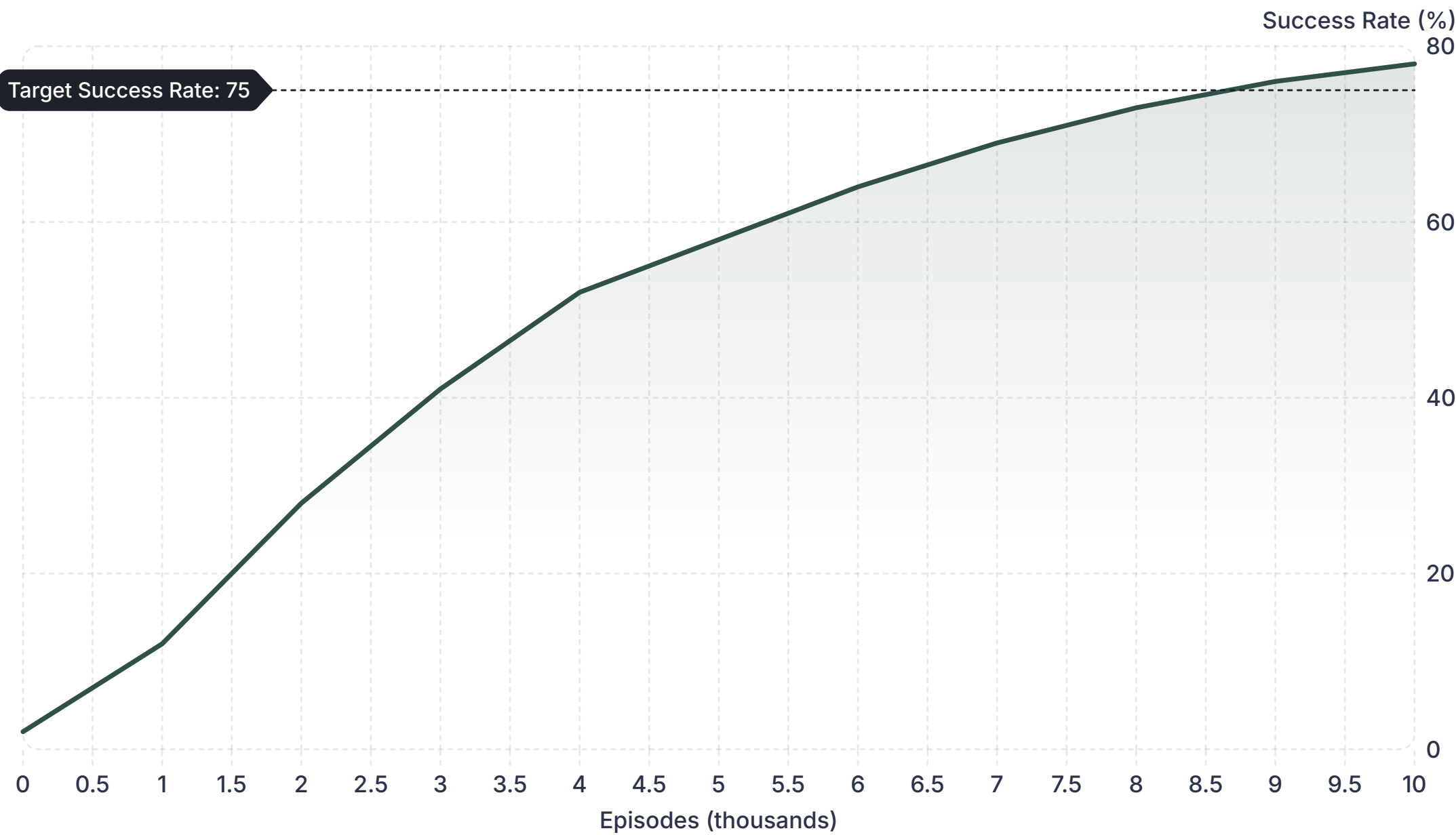
Q-Learning on FrozenLake: A Walkthrough

FrozenLake is a 4×4 grid world where the agent must navigate from start (S) to goal (G) while avoiding holes (H). The surface is slippery, making transitions stochastic — a perfect testbed for Q-learning.



Typical Q-Learning Training Results

Below is a representative training curve for Q-learning on the FrozenLake environment, showing how the agent's success rate improves over episodes as the Q-table converges.



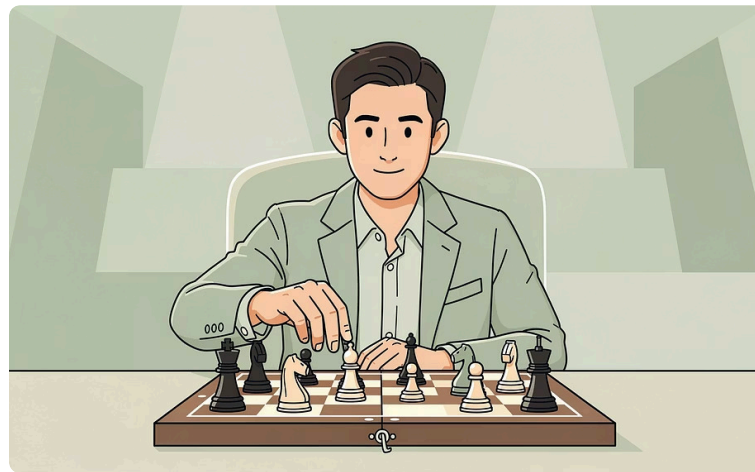
Early episodes show near-random performance as the agent explores. By 5,000 episodes, meaningful patterns emerge. The agent surpasses the 75% target around episode 8,000, demonstrating how iterative Q-value updates gradually converge toward the optimal policy.

Exploration vs. Exploitation



Exploration

The agent tries new, potentially suboptimal actions to discover better strategies. Without exploration, the agent may get stuck in a local optimum, never finding the truly best path.



Exploitation

The agent leverages what it already knows to maximize immediate reward. Pure exploitation means always choosing the current best action — efficient but potentially shortsighted.

- ❏ **The ϵ -greedy strategy** balances both: with probability ϵ the agent explores randomly, and with probability $(1-\epsilon)$ it exploits its current best action. Decaying ϵ over time shifts the balance from exploration to exploitation as the agent gains confidence.

UNLOCK

SUMMARY

Key Takeaways

Let's recap the essential concepts from this module on reinforcement learning fundamentals.

RL Paradigm

An agent learns by interacting with an environment, receiving rewards, and optimizing a policy — no labeled data required.

MDPs

The formal framework (S, A, P, R) with the Markov property enables tractable decision-making algorithms.

Q-Learning

An off-policy algorithm that learns optimal action-values through iterative updates and ϵ -greedy exploration.

Gym Toolkit

OpenAI Gym provides standardized environments to experiment with, benchmark, and validate RL algorithms hands-on.



UNLOCK

NEXT STEPS

Where to Go From Here



Practice with Gym

Implement Q-learning on FrozenLake and CartPole. Experiment with different hyperparameters (α , γ , ϵ decay) and observe how they affect convergence.



Deep Q-Networks (DQN)

When state spaces become too large for tables, neural networks approximate the Q-function. DQN was the breakthrough that mastered Atari games.



Policy Gradient Methods

Explore algorithms like REINFORCE and PPO that directly optimize the policy instead of learning value functions — essential for continuous action spaces.